

PROJECT STATEMENT

Smart Student Expense Tracker — Localized Personal Finance Management System

COURSE TITLE

[Course Name]

COURSE CODE

[Course Code]

STUDENT NAME

[Your Name]

REGISTRATION NO.

[Reg. Number]

1 PROBLEM STATEMENT

College students frequently encounter challenges in managing personal finances due to frequent micro-transactions across mess halls, academic supplies, transportation, utilities, and casual recreation. Without systematic logging, small daily expenditures rapidly accumulate, leading to budgetary overruns and financial anxiety before academic terms end.

Existing commercial personal finance platforms suffer from significant drawbacks: they are heavily monetized with advertisements, mandate cloud account registrations, require active internet connections, and overwhelm users with complex feature sets like stock tracking and credit score monitoring. There is a clear need for a dedicated, lightweight, local desktop application that enables students to instantly record, inspect, aggregate, and manage daily expenses without privacy risks or network dependencies.

2 SCOPE OF THE PROJECT

✓ INCLUDED IN SCOPE

- **Desktop GUI Interface:** Built natively using Python's Tkinter framework for rapid interaction.
- **In-Memory Data Engine:** Dynamic list storage holding Date, Category, Description, and Amount in INR (₹).
- **Core CRUD Operations:** Record insertion, sequential itemized listing, index-based deletion, and full memory clearing.
- **Real-Time Aggregation:** Automated arithmetic sum calculation across all recorded entries.
- **Validation & Error Alerting:** Dialog popups preventing blank entries and out-of-bounds array deletions.

✗ OUT OF SCOPE

- **External Database Systems:** Relational (SQL) or document (NoSQL) database integration.
- **Cloud Synchronization:** Multi-device synchronization or remote web server hosting.
- **Banking API Integration:** Direct automatic fetching of live transaction data from bank feeds.
- **Multi-Currency Engines:** Real-time foreign exchange rate lookup or multi-currency conversions.
- **Biometric Security:** Hardware-level authentication or complex multi-factor login systems.

3 TARGET USER PERSONAS

University Students Undergraduate and postgraduate students requiring a quick, low-friction tool to record day-to-day micro-expenses like snacks, books, and prints.	Hostel Boarders Students living on campus managing split utility payments, shared mess bills, laundry costs, and local transportation charges.	Budget-Conscious Users Individuals seeking zero-ad, privacy-focused software that runs entirely offline on local systems without cloud tracking.
--	--	--

4 HIGH-LEVEL FUNCTIONAL FEATURES

Feature Module	Functional Capability	User Interaction Mode
1. Transaction Entry	Captures Date, Category, Description, and Monetary Amount (₹) with type verification.	Form fields + "Add Expense" button
2. Itemized Display	Generates a formatted sequential list of all logged transactions assigned with 1-based index tags.	Modal popup via "Show Expenses"
3. Total Aggregation	Instantly computes the floating-point sum of all current expense amounts stored in memory.	Modal popup via "Total Expense"
4. Selective Deletion	Removes a specific transaction record using its corresponding 1-based display index number.	Index entry + "Delete Expense" button
5. Bulk Data Reset	Wipes all stored records from memory array in a single operation, resetting the tracker.	Control button "Clear All Expenses"
6. Validation Engine	Screens inputs for missing values and verifies numeric bounds prior to processing requests.	Automated `tkinter.messagebox` alerts

5 ACADEMIC & SYLLABUS ALIGNMENT STATEMENT

VITYarathi Technical Guidelines Compliance

This project strictly satisfies all core academic expectations for the VITYarathi flipped course evaluation:

- **Functional Requirements:** Incorporates three major functional modules (Data Input, Analytics/Reporting, and Storage Management).
- **Non-Functional Requirements:** Enforces rapid execution performance (<50ms), intuitive usability, localized reliability, and structured error handling.
- **Architecture:** Features an event-driven graphical modular architecture separating the GUI layout from event handler logic.